ALLIANCE UNIVERSITY

CAPSTONE REPORT

Front End Frame Works & Technologies

PROJECT TITLE

MEDISTOCK

Pharmacy Stock and Medicine Schedule Management System

SUBMITTED BY

Darshan HD PROV/ASAC/MCA/7/26/085

BasavanaGowda PROV/ASAC/MCA/7/26/084

Sunil K PROV/ASAC/MCA/7/26/014

Charan S PROV/ASAC/MCA/7/26/092

HOD: Dr J Lenin

Faculty: Raghav K



080-46199156
Alliance University , Chandapura
Anekal Main Road, Anekal
www.alliance.edu.in

Capstone Project Report

Master of Computer Application
Semester – III

Web Technologies

Project title: MEDISTOCK
PHARMACY STOCK AND MEDICINE SCHEDULE MANAGEMENT SYSTEM

Submitted By:

1. **Darshan HD (PROV/ASAC/MCA/7/26/085)**
2. **BasavanaGowda (PROV/ASAC/MCA/7/26/084)**
3. **Sunil K (PROV/ASAC/MCA/7/26/014)**
4. **Charan S (PROV/ASAC/MCA/7/26/092)**

Submitted To:

Faculty Name: **Raghav K** HOD Sign: _____

Faculty Signature: _____

Department of Computer
Application Alliance University
Chandapura - Anekal Main Road,
Anekal Bengaluru - 562 106

Oct- 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Raghav K, for his valuable guidance, encouragement, and continuous support throughout the development of this project. His constructive feedback, technical knowledge, and suggestions were instrumental in successfully completing this project.

I am also thankful to the Head of the Department and all the faculty members of the Department of Computer Applications for providing the necessary resources, laboratory facilities, and encouragement required to complete this project successfully.

I would also like to express my heartfelt thanks to my family and friends for their constant motivation, encouragement, and support throughout the course of this project.

Finally, I am grateful to everyone who directly or indirectly contributed to the successful completion of this project.

— Darshan HD

TABLE OF CONTENTS

Sl. No.	Particulars
1	About Our Course – Web Technologies
2	System Requirements
3	Introduction of Project
4	Objective / Aim
5	Implementation / Code with Explanation
6	Output Screenshots / Images
7	Future Enhancement
8	Conclusion
9	Reference Books / URLs
10	GitHub Link / Hosted Link

1. ABOUT OUR COURSE – WEB TECHNOLOGIES

1.1 Introduction to Web Technology

Web Technology refers to the collection of technologies, programming languages, frameworks, protocols, and tools used to develop websites and web-based applications. It is one of the most important areas of modern computer science because web applications are widely used in education, banking, healthcare, business, entertainment, and communication.

A web application generally consists of a front-end, a back-end, and a database. The front-end provides the interface through which users interact with the application. The back-end processes requests, performs business operations, manages authentication, and communicates with the database.

The Web Technology course gives students both theoretical and practical knowledge of developing web applications. It explains how a browser communicates with a server and how different technologies work together to provide a complete web-based solution.

1.2 Importance of Web Technology

- **Accessibility:** Web applications can be used from computers, tablets, and smartphones.
- **Platform independence:** Applications run on different operating systems through a browser.
- **Easy maintenance:** Updates are made on the server without reinstalling anything on each user's device.
- **Scalability:** Applications can be extended with additional users and features.
- **Integration:** Applications can communicate with other services through APIs.
- **Centralized data:** User and business data can be managed in a central database.

1.3 Front-End Technologies

HTML (HyperText Markup Language) creates the structure of a web page using headings, forms, tables, buttons, and input fields.

CSS (Cascading Style Sheets) controls the appearance and layout of pages: colours, fonts, spacing, grids, and responsive behaviour.

JavaScript makes pages interactive. It handles user actions, validates input, calls APIs with the Fetch API, and updates page content dynamically.

1.4 Back-End Technologies

Back-end development handles authentication, business logic, database operations, and API processing. **Node.js** is a JavaScript runtime that executes JavaScript on the server. **Express.js** is a Node.js framework that provides routing, middleware, and request/response handling. In the MediStock project, Node.js and Express.js implement the REST APIs and the server-side business rules.

1.5 Database and APIs

A database stores and manages application data. Common choices include MySQL, PostgreSQL, MongoDB, and SQLite. MediStock uses **SQLite**, a lightweight relational database, accessed through the SQLite module built into Node.js. An **API** (Application Programming Interface) lets software components communicate; REST APIs use HTTP methods such as GET, POST, PUT, and DELETE to exchange JSON data between the front end and the back end.

1.6 Learning Outcomes

1. Understanding the structure of web pages using HTML.
2. Designing interfaces using CSS.
3. Developing interactive functionality using JavaScript.
4. Understanding client-server communication.
5. Developing REST APIs using server-side technologies.
6. Connecting web applications with databases.
7. Implementing authentication and role-based authorization.
8. Applying web technology concepts to real-world projects.

The MediStock project applies these outcomes by combining front-end pages, a REST back-end, a relational database, JWT authentication, and role-based access control into a single pharmacy application.

1.7 Summary

Technologies such as HTML, CSS, JavaScript, Node.js, Express.js, and databases work together to create complete web solutions. Through this course and the development of MediStock, concepts of client-server architecture, database design, authentication, and API communication were practically understood and implemented.

2. SYSTEM REQUIREMENTS

2.1 Introduction

System requirements define the hardware and software resources needed to develop, run, test, and maintain the MediStock web application. Since MediStock is a full-stack application, the environment must support a browser, a code editor, a Node.js server, and a database file.

2.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or more
Storage	5 GB free space	10 GB or more
Display	1366 × 768	1920 × 1080
Internet	Needed for installing packages	Broadband

2.3 Software Requirements

Software	Purpose
Operating System	Windows 10 or later, Linux, or macOS
Visual Studio Code	Code editor for HTML, CSS, JavaScript, and Node.js
Node.js (v22.5 or higher)	Runs the Express server; its built-in `node:sqlite` module provides the database
npm	Installs and manages dependencies
Express.js 4	Web framework for routing and REST APIs
SQLite	Relational database stored in a single file (`medistock.db`)
bcryptjs	Password hashing
jsonwebtoken	Creating and verifying JWT tokens
dotenv	Loading secrets from the `.env` file
Web Browser	Chrome, Edge, or Firefox
Git and GitHub	Version control and sharing of source code

2.4 Functional Requirements

1. Patient self-registration.
2. Login for four roles: admin, pharmacist, cashier, and patient.
3. Password hashing and JWT-based authentication.
4. Role-based access control on every protected API.
5. Public medicine catalogue with search and stock status.

6. Adding new medicines with price and reorder level.
7. Adding stock batches with batch number, expiry date, and quantity.
8. Automatic stock deduction using the First-Expiry-First-Out (FEFO) rule.
9. Billing of one or more medicines in a single transaction.
10. Saving a medicine schedule (dose, times, food instructions, days) for a patient.
11. Alerts for low stock, medicines expiring within 90 days, and expired batches.
12. Patient dashboard showing the next dose, today's timings, and instructions.
13. Adding patients by pharmacy staff.

2.5 Non-Functional Requirements

- **Performance:** Normal operations and API calls respond quickly using a lightweight embedded database.
- **Security:** Passwords are stored as bcrypt hashes; secrets are kept in environment variables; protected routes require a valid token and an allowed role.
- **Usability:** A simple, clean interface with separate pages for patients and staff.
- **Reliability:** Billing runs inside a database transaction, so a failure never leaves partial data. Errors are returned as JSON instead of crashing the server.
- **Maintainability:** Code is organised into routes, controllers, services, middleware, and configuration.
- **Compatibility:** Works on modern browsers and common operating systems.

2.6 Summary

MediStock can be developed and run on an ordinary modern computer using freely available tools, which makes it suitable for academic development and small-scale deployment.

3. INTRODUCTION OF PROJECT

3.1 Project Overview

MediStock is a web-based pharmacy management system that helps a medical store manage its medicine stock and helps patients follow their prescribed medicine schedule. It combines two needs in one application: **inventory control** for the pharmacy (stock, batches, expiry, billing) and **medication reminders** for the patient (dose timings, food instructions, doctor and hospital details).

The tagline of the application is *"Buy Medicines and Never Miss a Dose."* Staff members record a sale and the tablet timings in one step, and the patient later sees a personal dashboard with the next dose and instructions.

3.2 Problem Statement

Small pharmacies and their patients face several problems:

- Stock is often tracked manually, so shortages are noticed late.

- Medicines pass their expiry date unnoticed, causing financial loss and safety risk.
- Older batches are not always sold before newer ones.
- Billing and stock updates are done separately, causing mismatches.
- Patients forget dose timings, food instructions, and course duration.
- Different staff roles have no controlled access to functions.

3.3 Proposed Solution

MediStock provides a central system in which the pharmacist adds medicines and batches, the system warns about low and expiring stock, and billing automatically reduces stock from the batch that expires first. When a medicine is sold to a registered patient, the staff member can also save the dose schedule. The patient logs in to a personal dashboard that lists today's doses, shows the next dose, and displays instructions until the course ends.

3.4 Need for the Project

1. Prevents sale of expired medicines.
2. Reduces wastage through FEFO stock usage.
3. Gives early warning of low stock.
4. Keeps billing and stock consistent through transactions.
5. Improves patient adherence to prescribed courses.
6. Restricts sensitive operations to authorised staff.

3.5 Scope of the Project

The scope includes user registration and login, role-based access, medicine and batch management, FEFO billing, stock alerts, schedule creation, and a patient dashboard. Future versions can add SMS reminders, reports, and payment integration.

3.6 Technologies Used

Technology	Use in the project
HTML5	Structure of the pages (home, login, staff, patient dashboard)
CSS3	Styling, grid layout, cards, and responsive design
JavaScript	Page logic and API communication using the Fetch API
Node.js	Server-side runtime
Express.js	REST API routing and middleware
SQLite (`node:sqlite`)	Relational database for users, medicines, batches, sales, and schedules
bcryptjs	Password hashing
JSON Web Token	Stateless authentication

Technology	Use in the project
dotenv	Environment variable management

3.7 Major Modules

1. Authentication and Registration.
2. Role-Based Authorization.
3. Medicine Catalogue.
4. Batch and Stock Management.
5. FEFO Stock Service.
6. Billing and Prescription Schedule.
7. Alerts (low stock, expiring, expired).
8. Patient Dashboard.

3.8 User Roles

Role	Permissions
Admin	Add medicines and batches, add patients, view alerts, bill customers
Pharmacist	Add medicines and batches, add patients, view alerts, bill customers
Cashier	View patients and stock, bill customers
Patient	Register, log in, and view personal medicine dashboard

3.9 Working Principle

User → Browser (HTML/CSS/JavaScript) → Fetch API → Express Server → Middleware (JWT + Role check) → Controller → Service → SQLite Database → JSON Response → Browser

The browser sends a request with the JWT token in the Authorization header. The server verifies the token and role, runs the controller logic, reads or writes the SQLite database, and returns JSON that the page displays.

3.10 Benefits of the Project

- Centralised stock and patient information.
- Lower expiry-related losses.
- Accurate and consistent billing.
- Better medicine adherence for patients.
- Secure, role-based access.
- Lightweight and easy to deploy.

3.11 Summary

MediStock solves practical problems of a small pharmacy by combining stock control, safe billing, and patient medication reminders in one web application.

4. OBJECTIVE / AIM

4.1 Aim of the Project

The aim of MediStock is to develop a secure web-based system that helps pharmacies manage medicine stock and expiry, bill customers accurately, and help patients follow their medicine schedule.

4.2 Objectives

1. **Secure authentication:** Provide registration and login with hashed passwords and JWT tokens.
2. **Role-based access:** Allow each role to use only the functions meant for it.
3. **Medicine management:** Store medicines with price and reorder level.
4. **Batch management:** Track batch number, expiry date, and quantity.
5. **FEFO stock usage:** Always sell stock that expires first and never sell expired stock.
6. **Safe billing:** Ensure a multi-medicine bill is saved completely or not at all.
7. **Alerts:** Warn about low, expiring, and expired stock.
8. **Medicine scheduling:** Save dose, timing, food instructions, doctor, and hospital for each patient.
9. **Patient dashboard:** Show the next dose, today's timings, and active instructions.

4.3 Educational Goals

The project demonstrates practical implementation of front-end development, back-end development, REST APIs, database design and SQL, authentication and authorization, transactions, and modular code organisation.

5. IMPLEMENTATION / CODE WITH EXPLANATION

5.1 Introduction

MediStock is implemented as a full-stack web application using HTML, CSS, JavaScript, Node.js, Express.js, SQLite, bcryptjs, and JSON Web Tokens. The front end provides four pages: the home page with the medicine catalogue, the login and registration page, the staff page, and the patient dashboard. The back end exposes REST APIs for authentication, medicines, billing, and the dashboard.

The code follows a modular architecture. Routes map URLs to controllers, controllers contain the request logic, services contain reusable business rules (such as FEFO), middleware handles authentication and authorization, and the config folder holds the database connection.

5.2 Project Structure

```
medistock/
├── config/
│   └── db.js                    Database connection and schema loading
├── controllers/
│   ├── auth.controller.js       Register, login, patient list, add patient
│   ├── billing.controller.js    Sale + schedule creation (transaction)
│   ├── dashboard.controller.js  Patient dashboard data
│   └── medicine.controller.js   Medicines, batches, alerts
├── Database/
│   ├── schema.sql               Table definitions
│   ├── seed.js                  Demo users, medicines and batches
│   └── medistock.db             SQLite database file
├── middleware/
│   ├── auth.js                  JWT verification
│   └── role.js                  Role-based access check
├── public/
│   ├── css/style.css            Common styles
│   ├── js/api.js                Helper functions for API calls
│   ├── index.html               Home page and medicine catalogue
│   ├── login.html               Login and registration
│   └── pages/
│       ├── staff.html           Staff page (billing, stock, alerts)
│       └── patient-dashboard.html
├── routes/                      auth, medicine, billing, dashboard routes
├── services/
│   ├── fefo.service.js          First-Expiry-First-Out stock deduction
│   └── helpers.js               Date helper
├── .env                         PORT and JWT_SECRET
├── server.js                    Application entry point
└── package.json                 Metadata and dependencies
```

5.3 Server Setup

The server is created with Express. It reads configuration from `.env`, parses JSON request bodies, serves the front-end files from the `public` folder, and mounts one router per module.

```
require('dotenv').config();
const express = require('express');
const app = express();

app.use(express.json());
app.use(express.static('public'));

app.use('/api/auth', require('./routes/auth.routes'));
app.use('/api/medicines', require('./routes/medicine.routes'));
app.use('/api/billing', require('./routes/billing.routes'));
app.use('/api/dashboard', require('./routes/dashboard.routes'));
```

A global error handler returns any thrown error as JSON, so the front end always receives a readable message.

```
app.use(function (err, req, res, next) {
  res.status(400).json({ error: err.message });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, function () {
  console.log('Open http://localhost:' + PORT);
});
```

5.4 Database Design

The database has five tables. The schema is stored in `schema.sql` and is executed every time the server starts, using `CREATE TABLE IF NOT EXISTS`.

Table	Purpose	Main columns
users	All accounts	id, name, email (unique), password (hash), role, phone
medicines	Medicine catalogue	id, name, price, reorder_level
batches	Stock lots of a medicine	id, medicine_id, batch_no, expiry_date, qty
sales	Record of every sale	id, patient_id, medicine_id, qty, total, created_at
schedules	Dose timings shown to patient	id, patient_id, medicine_id, dose, times, food, notes, start_date, days, doctor, hospital

```
CREATE TABLE IF NOT EXISTS batches (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  medicine_id INTEGER, batch_no TEXT, expiry_date TEXT, qty INTEGER
);
```

One medicine can have many batches, which makes it possible to track a different expiry date for each lot of stock.

5.5 Database Connection

The application uses the SQLite module that is built into Node.js, so no separate database server is required.

```
const { DatabaseSync } = require('node:sqlite');
const fs = require('fs');

const db = new DatabaseSync('database/medistock.db');
db.exec(fs.readFileSync('database/schema.sql', 'utf8'));
// Add doctor and hospital columns. If they already exist, the error is ignored.
```

```
try { db.exec('ALTER TABLE schedules ADD COLUMN doctor TEXT'); } catch (e) {}
try { db.exec('ALTER TABLE schedules ADD COLUMN hospital TEXT'); } catch (e) {}

module.exports = db;
```

The file opens the database, creates the tables if they are missing, and adds the `doctor` and `hospital` columns to older databases. The exported `db` object is shared by all controllers.

5.6 Seed Data

The `seed.js` script inserts demo data so that the application can be tested immediately. It creates four users (admin, pharmacist, cashier, and patient) with the password `pass123` stored as a bcrypt hash, four medicines, and five batches. One batch is deliberately already expired, and another is low in stock and close to expiry, so that the alerts can be demonstrated.

```
const password = bcrypt.hashSync('pass123', 10);
addUser.run('Priya Pharmacist', 'pharmacist@medistock.com', password, 'pharmacist', '');

addMed.run('Paracetamol 500mg', 2.5, 50);
addBatch.run(1, 'PCM-000', daysFromNow(-10), 20); // already expired
addBatch.run(2, 'AMX-014', daysFromNow(45), 8);   // low stock and expiring soon
```

5.7 User Authentication

Passwords are never stored as plain text. During registration the password is hashed with bcrypt, and the new account is always created with the role `patient`, so nobody can register themselves as staff.

```
const hash = bcrypt.hashSync(password, 10);
const result = db.prepare(
  'INSERT INTO users (name, email, password, role, phone) VALUES (?, ?, ?, ?, ?)'
).run(name, email, hash, 'patient', phone || '');
```

During login, the entered password is compared with the stored hash. If it matches, a JWT token that is valid for 12 hours is returned together with basic user information.

```
function makeToken(user) {
  return jwt.sign(user, process.env.JWT_SECRET, { expiresIn: '12h' });
}

exports.login = function (req, res) {
  const user = db.prepare('SELECT * FROM users WHERE email = ?').get(req.body.email || ''
);
  if (!user || !bcrypt.compareSync(req.body.password || '', user.password)) {
    return res.status(401).json({ error: 'Wrong email or password.' });
  }
  const info = { id: user.id, name: user.name, role: user.role };
  res.json({ token: makeToken(info), user: info });
```

```
};
```

All database queries use prepared statements with `?` placeholders, which protects the application against SQL injection.

5.8 Authentication and Role Middleware

Protected routes pass through two middleware functions. The first verifies the JWT sent in the `Authorization: Bearer <token>` header and attaches the user to the request.

```
module.exports = function (req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.split(' ')[1];
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (e) {
    res.status(401).json({ error: 'Please log in again.' });
  }
};
```

The second function accepts a list of allowed roles and returns HTTP 403 for everyone else.

```
module.exports = function (...roles) {
  return function (req, res, next) {
    if (roles.includes(req.user.role)) {
      next();
    } else {
      res.status(403).json({ error: 'You are not allowed to do this.' });
    }
  };
};
```

The two are combined on each route, for example:

```
router.get('/alerts', auth, role('admin', 'pharmacist'), c.alerts);
router.post('/', auth, role('admin', 'pharmacist', 'cashier'), c.create);  // billing
router.get('/', auth, role('patient'), c.me);                              // dashboard
```

5.9 REST API Summary

Method	Endpoint	Allowed roles	Description
POST	/api/auth/register	Public	Register a new patient
POST	/api/auth/login	Public	Login and receive a token
GET	/api/auth/patients	Admin, Pharmacist, Cashier	List patients

Method	Endpoint	Allowed roles	Description
POST	/api/auth/patients	Admin, Pharmacist	Add a patient
GET	/api/medicines	Public	Medicine list with current stock
GET	/api/medicines/alerts	Admin, Pharmacist	Low, expiring, and expired stock
POST	/api/medicines	Admin, Pharmacist	Add a medicine
POST	/api/medicines/batches	Admin, Pharmacist	Add a stock batch
POST	/api/billing	Admin, Pharmacist, Cashier	Sell medicines and save schedule
GET	/api/dashboard	Patient	Patient profile and schedules

5.10 Medicine Stock Calculation

The medicine list shows the available stock of each medicine. A single SQL query joins medicines with batches and adds only those batches whose expiry date has not passed, so expired stock is never counted as available.

```
function getMedicines() {
  return db.prepare(
    'SELECT m.id, m.name, m.price, m.reorder_level, ' +
    'COALESCE(SUM(CASE WHEN b.expiry_date >= ? THEN b.qty END), 0) AS stock ' +
    'FROM medicines m LEFT JOIN batches b ON b.medicine_id = m.id ' +
    'GROUP BY m.id ORDER BY m.name'
  ).all(today());
}
```

The helper function `today()` returns the current date as `YYYY-MM-DD`, which allows dates to be compared as text.

5.11 FEFO Stock Service

FEFO means First-Expiry-First-Out. When a medicine is sold, the service selects the unexpired batches ordered by expiry date, checks that enough stock exists, and then reduces quantity from the earliest-expiring batch first, moving to the next batch when one becomes empty.

```
exports.takeStock = function (medicineId, qty) {
  const batches = db.prepare(
    'SELECT * FROM batches WHERE medicine_id = ? AND qty > 0 AND expiry_date >= ? ORDER B
Y expiry_date'
  ).all(medicineId, today());

  let available = 0;
  for (const b of batches) { available = available + b.qty; }
  if (available < qty) { throw new Error('Not enough unexpired stock.'); }

  let left = qty;
  for (const b of batches) {
```

```
    if (left === 0) break;
    const take = Math.min(b.qty, left);
    db.prepare('UPDATE batches SET qty = qty - ? WHERE id = ?').run(take, b.id);
    left = left - take;
  }
};
```

This ensures that older stock is sold first and that expired medicine can never be sold.

5.12 Billing with Transaction

The billing controller can sell several medicines in one bill. It validates every item, checks the dose times, reduces stock using the FEFO service, records the sale, and, when a patient is selected and times are given, saves a schedule for the patient.

```
db.exec('BEGIN');
try {
  for (const item of items) {
    const quantity = parseInt(item.qty);
    const med = db.prepare('SELECT * FROM medicines WHERE id = ?').get(Number(item.medici
ne_id));
    if (!med) throw new Error('Medicine not found.');

    takeStock(med.id, quantity);

    const total = med.price * quantity;
    grandTotal = grandTotal + total;
    db.prepare('INSERT INTO sales (patient_id, medicine_id, qty, total) VALUES (?, ?, ?,
?)')
      .run(patient, med.id, quantity, total);
    // ... schedule is inserted here when patient and times are given
  }
  db.exec('COMMIT');
} catch (e) {
  db.exec('ROLLBACK');
  throw e;
}
```

`BEGIN ... COMMIT` makes the whole bill atomic: if the second medicine is out of stock, the first sale is rolled back too, so stock and sales records always stay consistent. Dose times are validated with a regular expression so that only the format `08:00,14:00,21:00` is accepted.

```
if (!/^([01][0-9]|2[0-3]):[0-5][0-9]$/.test(t.trim())) {
  throw new Error('Write times like 08:00,14:00,21:00 (for ' + med.name + ').');
}
```

5.13 Stock Alerts

The alerts API returns three lists for the pharmacist: medicines whose stock is at or below the reorder level, batches that expire within the next 90 days, and batches that have already expired but still contain stock.

```
const lowStock = getMedicines().filter(function (m) {
  return m.stock <= m.reorder_level;
});
const expiring = db.prepare(
  "SELECT m.name, b.batch_no, b.expiry_date, b.qty FROM batches b JOIN medicines m ON m.i
d = b.medicine_id " +
  "WHERE b.qty > 0 AND b.expiry_date >= ? AND b.expiry_date <= date(?, '+90 days') ORDER
BY b.expiry_date"
).all(t, t);
```

5.14 Patient Dashboard API

The dashboard controller returns the logged-in patient's profile together with all of their schedules. The user id is taken from the verified token, so a patient can see only their own data.

```
exports.me = function (req, res) {
  const user = db.prepare('SELECT name, email, phone FROM users WHERE id = ?').get(req.us
er.id);
  const schedules = db.prepare(
    'SELECT s.*, m.name AS medicine FROM schedules s JOIN medicines m ON m.id = s.medicin
e_id ' +
    'WHERE s.patient_id = ? ORDER BY s.id DESC'
  ).all(req.user.id);
  res.json({ user: user, schedules: schedules });
};
```

5.15 Front-End and Back-End Communication

All pages use one helper function, `api()`, in `js/api.js`. It sends the request with the stored token and converts server errors into JavaScript errors. The `esc()` helper escapes HTML characters before data is displayed, which prevents cross-site scripting (XSS).

```
async function api(path, method, body) {
  const response = await fetch('/api/' + path, {
    method: method || 'GET',
    headers: {
      'Content-Type': 'application/json',
      Authorization: 'Bearer ' + localStorage.getItem('token')
    },
    body: body ? JSON.stringify(body) : undefined
  });
  const data = await response.json();
  if (!response.ok) { throw new Error(data.error || 'Something went wrong'); }
  return data;
```

```
}
```

After login, the token and user details are saved in `localStorage`, and the user is redirected to the staff page or the patient dashboard according to the role. The function `requireRole()` sends users back to the login page if they open a page that their role is not allowed to see.

5.16 Patient Dashboard Logic

The dashboard page calculates the last day of each course from `start_date` and `days`, hides finished courses, builds a list of all dose times, sorts them, and finds the next upcoming dose by comparing with the current time.

```
const lastDay = new Date(s.start_date + 'T00:00:00');
lastDay.setDate(lastDay.getDate() + s.days - 1);
if (lastDay < midnight) continue; // course is finished

doses.sort(function (a, b) { return a.time.localeCompare(b.time); });
let next = null;
for (const d of doses) {
  if (d.time >= nowText) { next = d; break; }
}
```

Each dose is shown as *Passed* or *Upcoming*, along with the medicine, dose, food instruction, doctor, and hospital.

5.17 Environment Configuration

Secrets are kept outside the source code in a `.env` file.

```
PORT=3000
JWT_SECRET=your_secret_key
```

5.18 Running the Project

```
npm install
npm run seed
npm start
```

The application opens at `http://localhost:3000`. Demo logins (password `pass123`): `pharmacist@medistock.com`, `cashier@medistock.com`, `patient@medistock.com`.

5.19 Implementation Summary

The implementation combines front-end pages, REST APIs, a relational database, secure authentication, role-based authorization, FEFO stock handling, and transaction-safe billing. The

modular structure keeps every responsibility in its own file, which makes the project easy to maintain and extend.

6. OUTPUT SCREENSHOTS / IMAGES

The screenshots below show the working MediStock application, from the public home page to the staff and patient dashboards.

Figure 6.1: MediStock Home Page and Medicine Catalogue

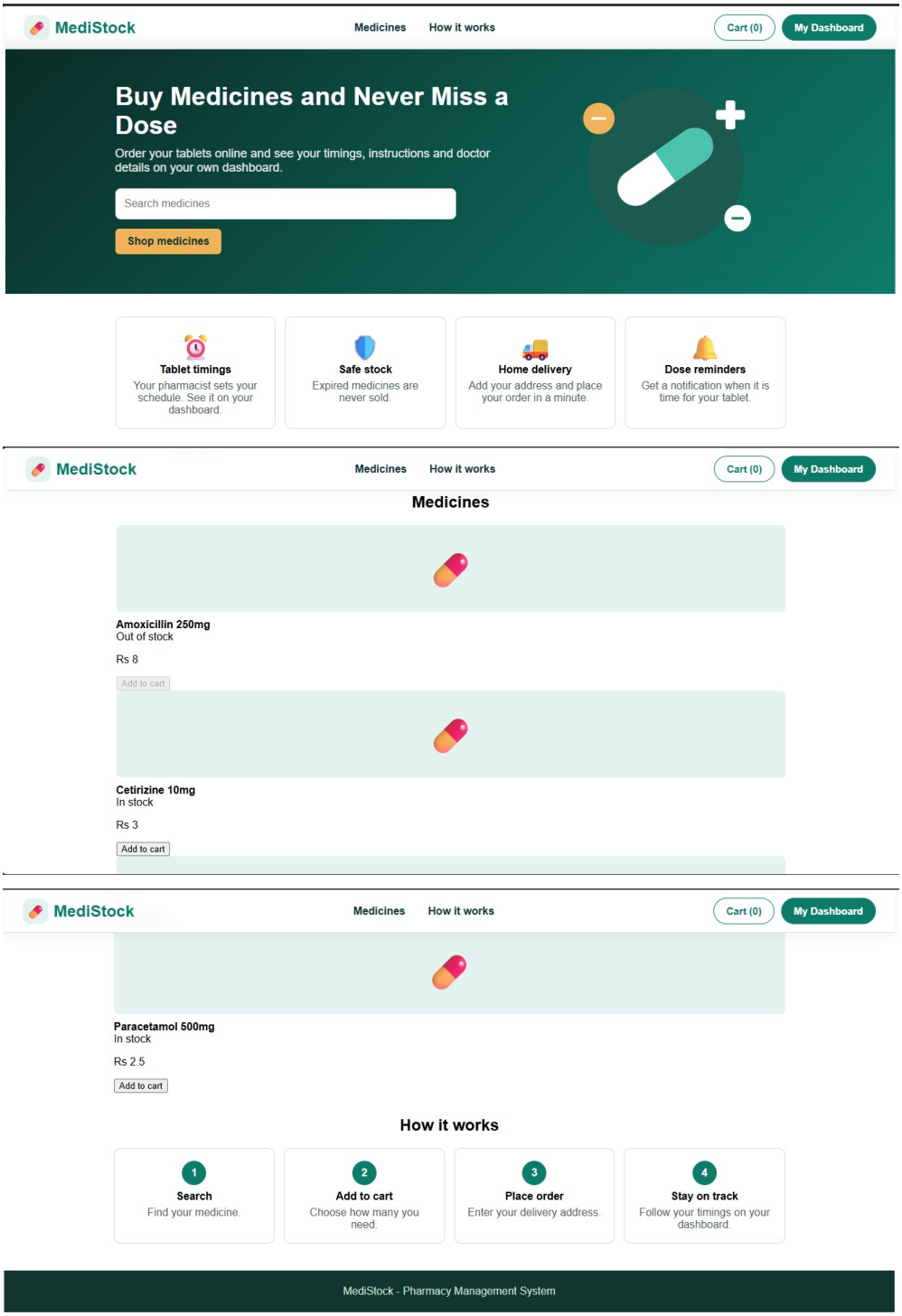


Figure 6.2: Login Page

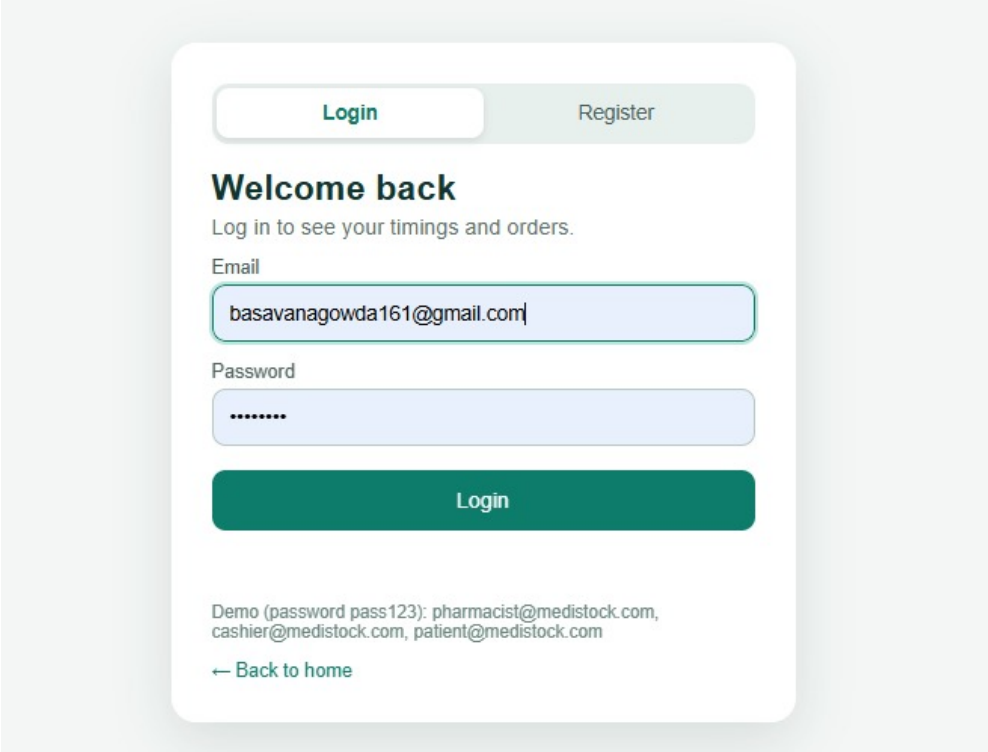


Figure 6.3: Patient Registration Page

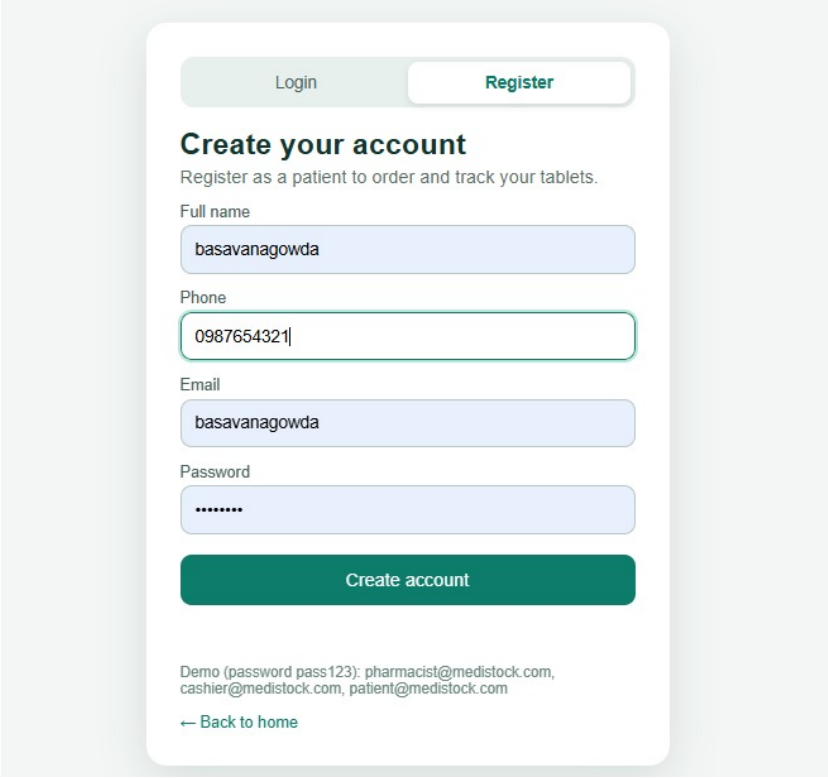


Figure 6.4: Staff Page – Add Patient

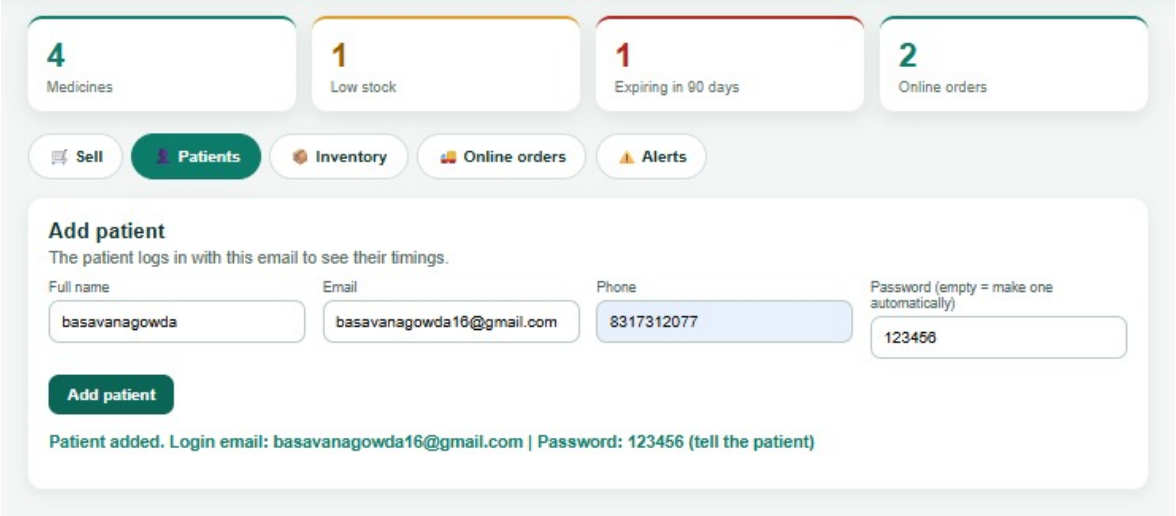


Figure 6.5: Staff Page – Sell Medicine with Schedule

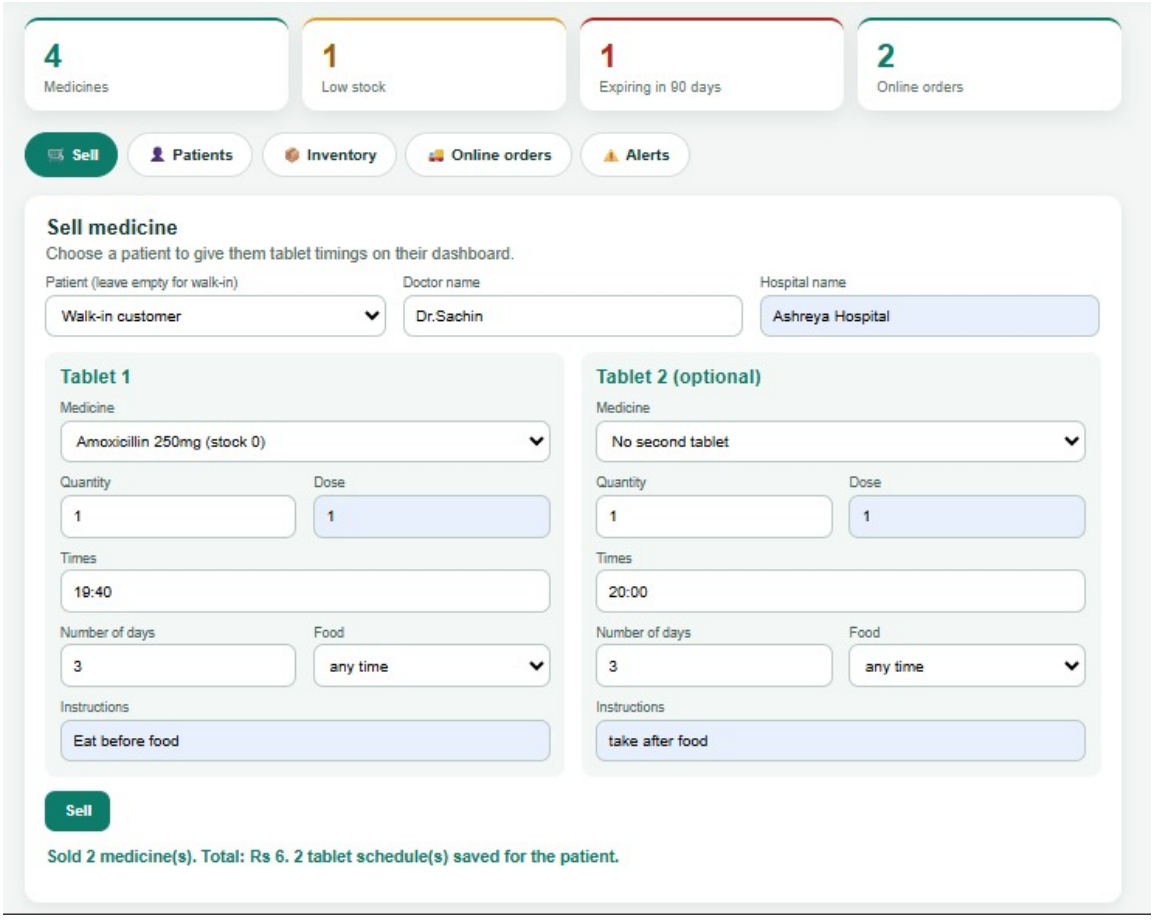


Figure 6.6: Staff Page – Add Medicine and Add Stock Batch

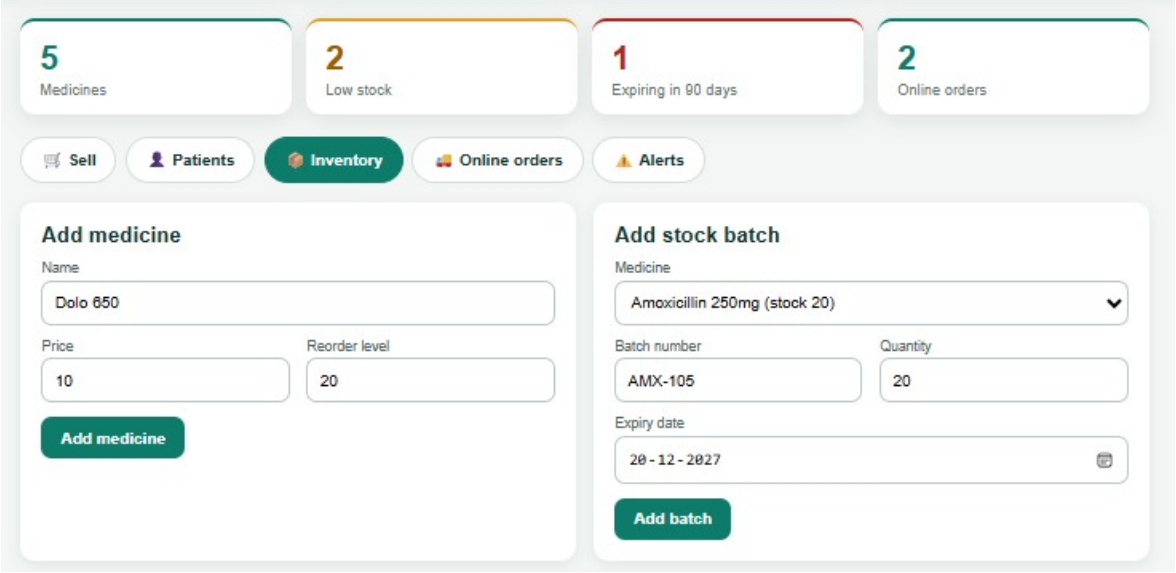


Figure 6.7: Stock Alerts

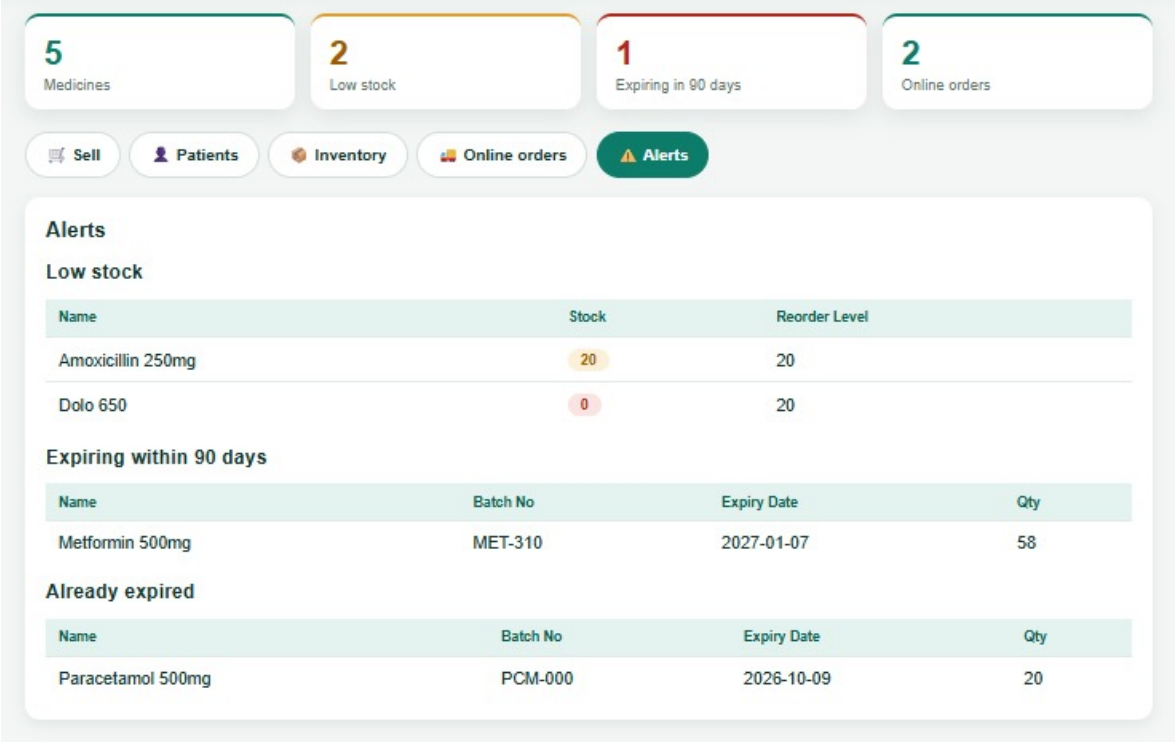


Figure 6.8: Stock Table



Stock

Name	Price	Stock	Reorder Level
Amoxicillin 250mg	8	20	20
Cetirizine 10mg	3	117	30
Dolo 650	10	0	20
Metformin 500mg	4	58	40
Paracetamol 500mg	2.5	190	50

Figure 6.9: Patient Dashboard

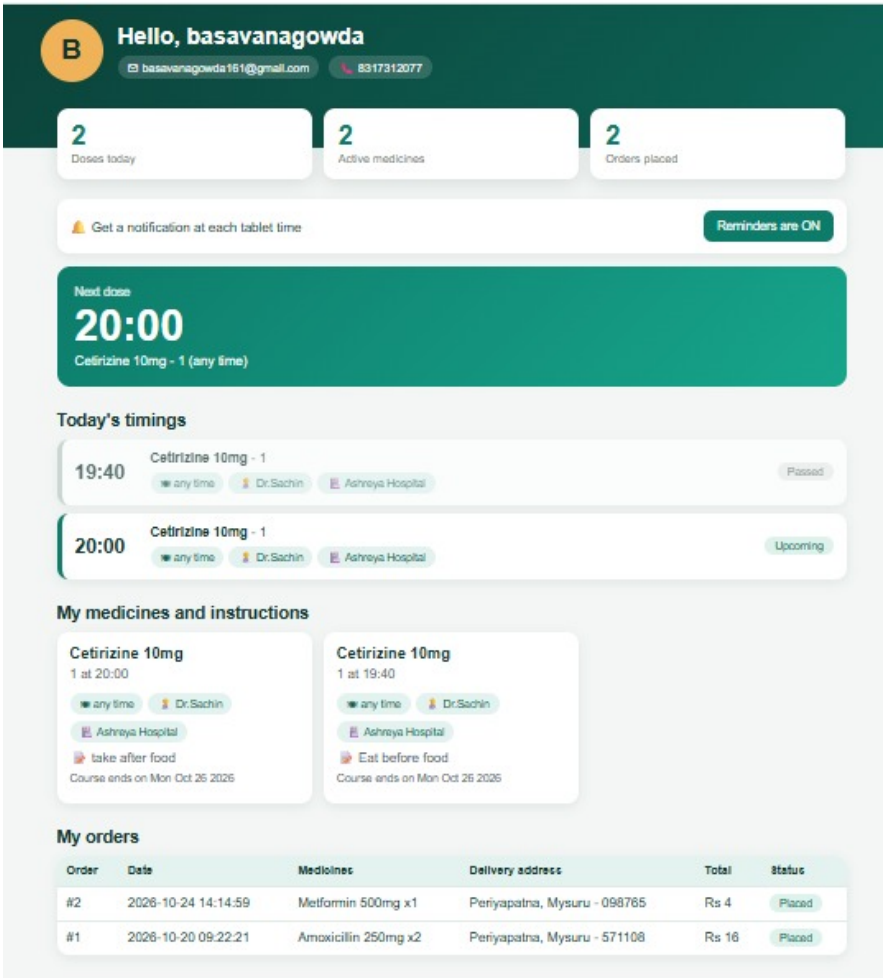
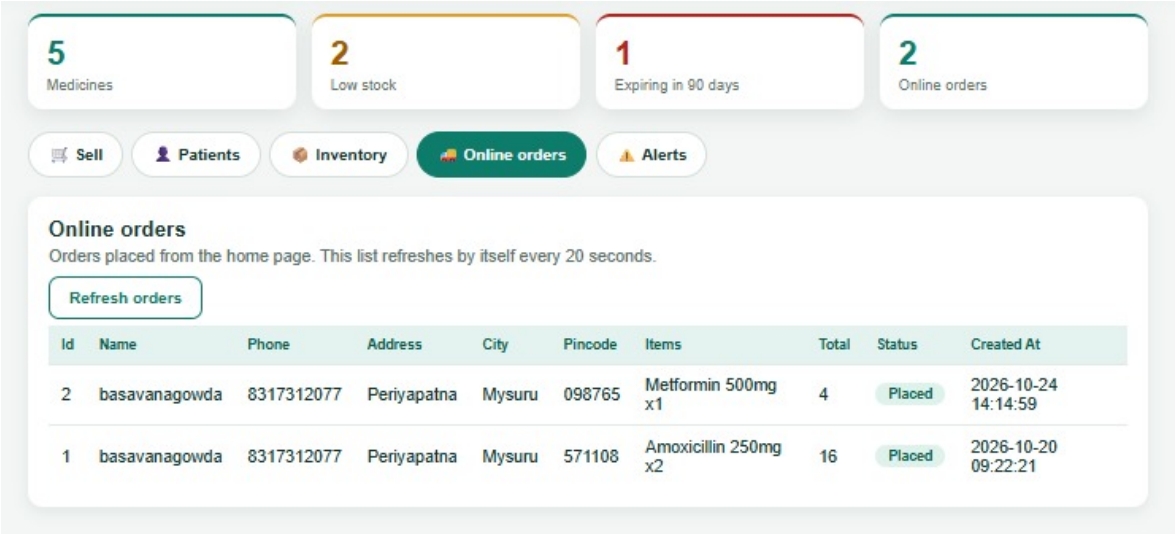


Figure 6.10: Access Denied for Wrong Role (optional)

[Insert screenshot of the 403 message or redirect to login when a patient tries to open a staff API]

Figure 6.11: Staff Page – Online Orders



7. FUTURE ENHANCEMENT

7.1 Introduction

MediStock already covers stock control, billing, and patient schedules. The following improvements can make it more complete and suitable for real pharmacies.

7.2 SMS, Email, and Push Reminders

The patient dashboard currently shows doses only when the page is opened. Future versions can send SMS, email, or mobile push notifications at the scheduled dose times and when a course is about to end.

7.3 Billing Improvements

- Printable invoices and PDF receipts.
- Discounts, GST/tax calculation, and payment method records.
- Online payment gateway integration.
- Sales history and refunds.

7.4 Reports and Analytics

Daily, weekly, and monthly sales reports, best-selling medicines, profit reports, and charts for stock movement can help the owner make better purchase decisions.

7.5 Supplier and Purchase Management

A supplier module can store supplier details, purchase orders, and goods received. Low-stock alerts could automatically create purchase suggestions.

7.6 Barcode and QR Scanning

Batch numbers and medicines could be scanned using a barcode reader or phone camera to make billing and stock entry faster.

7.7 Online Ordering and Home Delivery

Patients could order medicines online, upload prescriptions, and track delivery status.

7.8 Mobile Application

A mobile app using Flutter or React Native would allow patients to receive reminders and view schedules directly on their phones.

7.9 Prescription Upload and Interaction Warnings

Prescriptions could be uploaded as images or PDFs, and the system could warn about duplicate medicines or known drug interactions.

7.10 Security Improvements

- Rate limiting and account lockout after repeated failed logins.
- Email verification and password reset.
- Two-factor authentication.
- HTTP-only cookies for tokens, and HTTPS deployment.
- Audit log of stock and billing changes.

7.11 Database Scaling

For many branches or users, the SQLite database could be migrated to MySQL or PostgreSQL, and a multi-branch inventory feature could be added.

8. CONCLUSION

8.1 Conclusion

MediStock was developed as a web-based pharmacy management system that combines inventory control with patient medication scheduling. The project demonstrates the practical use of HTML, CSS, JavaScript, Node.js, Express.js, SQLite, JWT, and bcrypt to build a complete full-stack application.

The system helps pharmacies avoid selling expired medicine through FEFO stock handling, keeps billing and stock consistent through database transactions, and warns staff about low and expiring stock. At the same time, it helps patients follow their prescribed medicines through a personal dashboard.

8.2 Project Outcome

1. A full-stack pharmacy web application was developed.
2. User registration and login were implemented with hashed passwords.
3. JWT authentication and role-based authorization were implemented for four roles.
4. Medicine and batch management was provided.
5. FEFO-based stock deduction was implemented.
6. Transaction-safe multi-medicine billing was implemented.
7. Alerts for low, expiring, and expired stock were provided.
8. Medicine schedules with doctor and hospital details were saved for patients.
9. A patient dashboard with next-dose information was developed.
10. A modular and maintainable code structure was followed.

8.3 Learning Outcome

Developing MediStock gave practical experience in designing a relational database, writing REST APIs, using middleware for authentication and authorization, handling transactions, validating

input, preventing SQL injection and XSS, and connecting a JavaScript front end to a Node.js back end.

8.4 Final Conclusion

MediStock successfully shows how Web Technology can solve a real-world problem in the healthcare retail sector. With future enhancements such as reminders, reports, online ordering, and stronger security, it can grow into a complete pharmacy management platform.

9. REFERENCE BOOKS / URLs

9.1 Reference Books

1. Jon Duckett, *HTML and CSS: Design and Build Websites*, Wiley.
2. Jon Duckett, *JavaScript and JQuery: Interactive Front-End Web Development*, Wiley.
3. David Flanagan, *JavaScript: The Definitive Guide*, O'Reilly Media.
4. Ethan Brown, *Web Development with Node and Express*, O'Reilly Media.
5. Brad Dayley, Brendan Dayley and Caleb Dayley, *Node.js, MongoDB and Angular Web Development*, Addison-Wesley.

9.2 Online References

1. Node.js Documentation – https://nodejs.org/docs/
2. Node.js SQLite Module – https://nodejs.org/api/sqlite.html
3. Express.js Documentation – https://expressjs.com/
4. SQLite Documentation – https://www.sqlite.org/docs.html
5. MDN Web Docs – JavaScript – https://developer.mozilla.org/en-US/docs/Web/JavaScript
6. MDN Web Docs – HTML – https://developer.mozilla.org/en-US/docs/Web/HTML
7. MDN Web Docs – CSS – https://developer.mozilla.org/en-US/docs/Web/CSS
8. JSON Web Token Introduction – https://jwt.io/introduction
9. bcryptjs on npm – https://www.npmjs.com/package/bcryptjs
10. GitHub Documentation – https://docs.github.com/

10. GITHUB LINK / HOSTED LINK

10.1 GitHub Repository

[Paste your GitHub repository link here]

10.2 Hosted Project

[Paste your hosted link here, or write "Run locally at http://localhost:3000"]

